웹서비스설계 한국어 분반

과제 2

항목	작성 내용
학번과 이름	202413123 조이진
서비스명과 기술 조합	ApplyFlow | Node.js 24.x · Express 5 · SQLite
GitHub URL	https://github.com/zohijin-k/ApplyFlow
커밋 SHA	6f657b9938ce0ef135d563830bf6223c0b200745
실행 환경과 Node 버전	Node.js v24.22.3 | npm | macOS | 정상 엔진 검사

프로젝트 개요

ApplyFlow는 행사 신청과 운영자 승인을 처리합니다. 학생 신청, 본인 대기 취소, 담당 운영자의 직접 승인과 승인 취소를 제공합니다. 승인 취소로 생긴 자리는 같은 행사에서 가장 먼저 대기한 신청 한 건에 연결합니다. 모든 성공한 상태 변경은 변경 이력과 함께 SQLite에 저장합니다.

핵심 결과

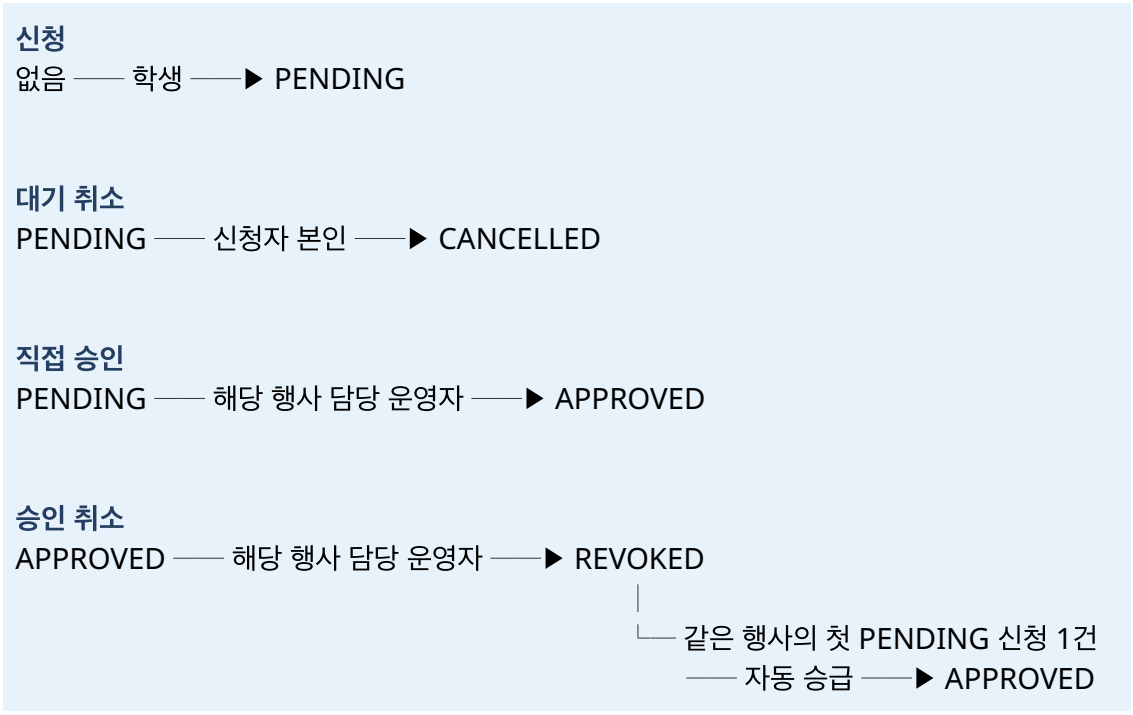
- 기반 테스트 4/4, 필수 T01~T32 32/32, 선택 심화 동시성 테스트 1/1 통과. 오류 주입 테스트를 포함해 실패 시 상태와 이력이 함께 롤백되는 것을 확인.
- U01~U05 실제 화면 흐름은 headless Chrome에서 확인 후 Screenshots 저장.

1 서비스와 업무 규칙

ApplyFlow는 학생의 행사 신청과 운영자의 승인 업무를 처리하는 웹서비스입니다. 학생은 행사에 신청하고 본인의 대기 신청을 취소할 수 있으며, 담당 운영자는 신청을 승인하거나 사유를 입력해 승인을 취소할 수 있습니다. 승인 취소로 자리가 생기면 같은 행사에서 가장 먼저 대기한 신청 한 건을 자동 승인합니다. 신청 상태와 변경 이력은 SQLite에 함께 저장하며, 처리 중 오류가 나면 두 데이터 모두 원래 상태로 되돌립니다.

규칙	구현 위치 또는 설명
R1 신원과 권한	• `src/auth.js`에서 JWT 서명 알고리즘·issuer·audience·만료를 확인 • 사용자 정보를 DB에서 다시 조회 • `src/application-service.js`의 `apply`, `cancel`, `approve`, `revoke`에서 학생, 신청자 본인, 행사 담당 운영자 권한 검사
R2 중복	• `apply()`에서 사용자·행사별 기존 신청 검사 • `src/db.js`의 `UNIQUE(event_id, user_id)` 제약으로 저장 단계에서도 중복을 막음 • 중복 제약 오류는 409 `ALREADY_APPLIED`로 반환 • 취소·승인 취소된 신청도 다시 신청할 수 없음
R3 상태	• `apply()`는 PENDING • `cancel()`은 PENDING→CANCELLED • `approve()`와 자동 승급은 PENDING→APPROVED • `revoke()`는 APPROVED→REVOKED • CANCELLED와 REVOKED는 종료 상태로 두고 잘못된 상태 변경을 409 `INVALID_STATE`로 거부
R4 정원	• `repository.js`의 `approvedCount()`로 APPROVED 신청만 계산 • `approve()`는 트랜잭션 안에서 정원을 다시 확인 • 정원이 찬 경우 409 `CAPACITY_FULL` 반환
R5 자동 승급	• `repository.js`의 `nextPending()`이 같은 행사의 PENDING 신청을 `created_at ASC, id ASC LIMIT 1`로 조회 • 승인 취소 요청마다 최대 한 명을 승급 | 대기자가 없으면 `promoted = null`
R6 사유와 이력	• `src/app.js`에서 승인 취소 사유를 앞뒤 공백 제거 후 5~200자로 검사 • 추가 본문 필드를 거부 • `addHistory()`는 행위자, 행위, 전후 상태, 사유, 원인 이력 ID, 생성 시각을 저장 • 자동 승급 이력은 해당 REVOKE 이력 ID를 원인으로 연결
R7 원자성	• `src/application-service.js`의 `transaction()`은 `BEGIN IMMEDIATE`로 시작해 정상 처리 시, `COMMIT` • 예외 발생 시, `ROLLBACK` • 승인 취소와 자동 승급 상태 변경 및 두 이력을 같은 트랜잭션에 포함
R8 최신 상태	• 각 변경 요청은 트랜잭션 안에서 신청과 행사 정보를 DB에서 다시 읽고 권한·현재 상태·정원을 확인 • 오래된 화면의 상태를 신뢰하지 않으며, 거부된 요청은 신청 상태와 이력을 변경하지 않음

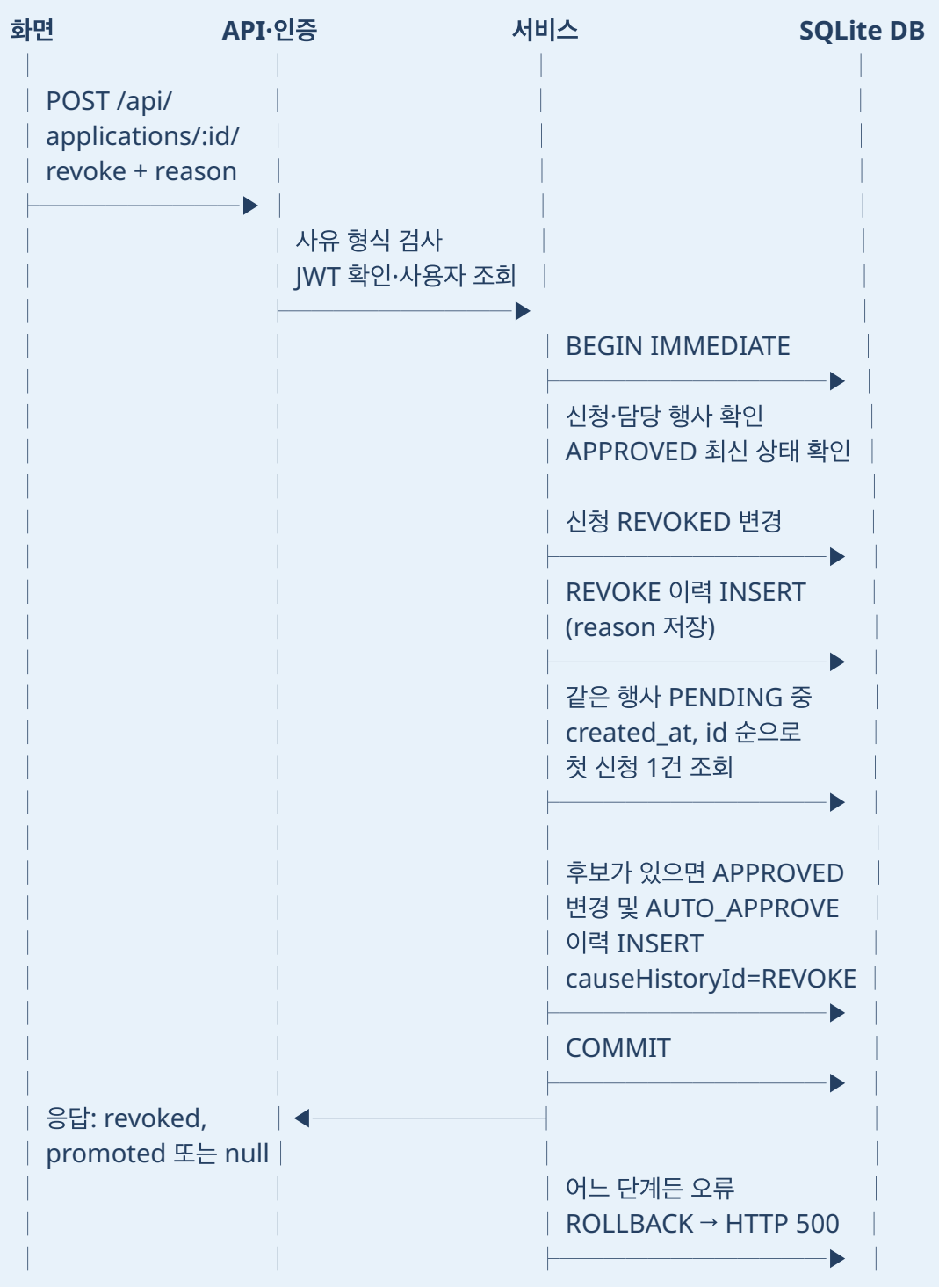
상태 전이도



학생이 APPROVED 신청을 직접 취소하거나 운영자가 다른 담당자의 행사 신청을 변경하는 전이는 허용하지 않습니다. CANCELLED와 REVOKED 상태에서 다시 신청하거나 상태를 변경하려는 요청도 거부합니다. 이 검사는 서버에서 DB의 최신 상태를 기준으로 수행하고, 거부된 요청에는 이력을 추가하지 않습니다.

2 승인 취소 흐름과 트랜잭션 설계

승인 취소 시퀀스 다이어그램



트랜잭션은 `src/application-service.js`의 `transaction(db, work)`에서 시작하고 끝냅니다. `BEGIN IMMEDIATE`로 쓰기 트랜잭션을 시작한 뒤 대상 신청과 권한·상태를 확인하고 상태 및 이력을 변경합니다. 각 단계가 성공하면 `COMMIT`하고, 예외가 발생하면 `ROLLBACK`합니다. 승인 취소에서는 REVOKE 상태·이력, 대기자 조회, 자동 승급 상태·이력을 한 트랜잭션에 포함합니다.

신청과 대기 취소

`src/application-service.js`의 `apply(actor, eventId)`는 행사 존재 여부와 학생 권한을 확인하고, `repo.existing(eventId, actor.id)`로 중복 신청을 검사합니다. 이후 신청을 PENDING으로 저장하고 APPLY 이력을 추가합니다. DB의 사용자·행사 UNIQUE 제약 위반도 409 ALREADY_APPLIED로 변환합니다.

`cancel(actor, applicationId)`는 신청을 다시 조회한 후 신청자 본인인지, 현재 상태가 PENDING인지 확인합니다. 조건을 만족하면 CANCELLED로 바꾸고 CANCEL 이력을 저장합니다. 대기 취소는 승인 자리를 비우지 않으므로 자동 승급을 실행하지 않습니다.

직접 승인과 정원

`approve(actor, applicationId)`는 신청과 행사를 조회하고 요청자가 해당 행사의 담당 운영자인지 확인합니다. 신청의 최신 상태가 PENDING인지 확인한 뒤 `repo.approvedCount(current.eventId)`와 행사 정원을 비교합니다. 정원이 찬 경우 409 CAPACITY_FULL로 거부하며, 승인 가능한 경우 APPROVED 변경과 APPROVE 이력을 같은 트랜잭션으로 저장합니다.

정원 조회에 사용하는 SQL은 다음과 같습니다.

```
SELECT COUNT(*) AS n
FROM applications
WHERE event_id = ?
  AND status = 'APPROVED';
```

승인 취소와 대기 순서

`revoke(actor, applicationId, reason)`는 담당 운영자와 APPROVED 상태를 확인한 후 신청을 REVOKED로 바꾸고 사유가 포함된 REVOKE 이력을 저장합니다. `repository.js`의 대기자 선택 SQL은 같은 행사에서 PENDING 상태인 신청만 조회하고, 등록 시각이 빠른 신청을 우선하며 시각이 같으면 ID가 작은 신청을 선택합니다.

```
SELECT ...
FROM applications a
WHERE a.event_id = ?
  AND a.status = 'PENDING'
ORDER BY a.created_at ASC, a.id ASC
LIMIT 1;
```

이 조회는 승인 취소 트랜잭션 안에서 수행합니다. 따라서 다른 행사 신청과 CANCELLED·REVOKED 신청은 후보에서 제외됩니다. 대기자가 없으면 `promoted`는 `null`로 반환하고 자리를 비운 채 종료합니다.

이력과 전체 롤백

이력 테이블에는 신청 ID, 행위자 ID, 행위, 변경 전·후 상태, 사유, 원인 이력 ID, 생성 시각을 저장합니다. AUTO_APPROVE 이력의 `causeHistoryId`는 바로 앞서 생성한 REVOKE 이력의 ID입니다. 승인 취소와 자동 승급 사이에 이력 저장 오류가 발생하면 REVOKE 변경·이력과 승급 상태·이력이 모두 롤백되어 요청 전 상태로 돌아갑니다.

제공된 코드는 로그인·JWT 인증, DB 스키마와 기본 화면, 조회 API, 사유 형식 검사 등입니다. 과제 구현 부분은 `src/application-service.js`의 `apply`, `cancel`, `approve`, `revoke`와 공통 트랜잭션 처리, `src/repository.js`의 대기자 조회 및 이력 저장 연동입니다. 기존 API 계약과 DB 스키마를 유지하면서 업무 규칙, 이력 저장 시점과 트랜잭션 범위를 구현했습니다.

3 검증 결과 T01부터 T08

HTTP와 code, 재조회한 상태·승인 인원·이력 건수와 내용을 기록합니다. 미실행은 그대로 표시하고 증거 파일 또는 그림 번호를 연결하세요.

각 테스트의 실제 HTTP 결과와 DB 상태·이력 확인은 `tests/assignment.test.js`에 있습니다. 전체 실행 결과는 `docs/evidence/test-results.md`에 기록했습니다.

ID와 상황	예상 결과	실제 결과와 데이터	증거 위치
T01 정상 신청	201 · 본인 ID · PENDING	PENDING201 반환. 로그인 학생 ID로 신청되고 PENDING 상태와 APPLY 이력이 저장됐습니다. Location 헤더가 반환됐습니다.	`tests/assignment.test.js` T01
T02 인증 누락·변조	401 · 데이터 불변	인증 누락·잘못된 토큰은 401로 거부됐고 신청 및 이력은 변하지 않았습니다.	`tests/assignment.test.js` T02
T03 신원·상태 위조 필드	400 · 데이터 불변	userId·role·status 등 허용되지 않은 본문 필드는 400으로 거부됐고 DB 변경이 없었습니다.	`tests/assignment.test.js` T03
T04 같은 행사 재신청	409 · 한 건 유지	이미 신청한 사용자의 재신청은 409 `ALREADY_APPLIED`로 거부됐고 신청·이력은 추가되지 않았습니다.	`tests/assignment.test.js` T04
T05 다른 행사 신청	201 · 별도 신청	다른 행사 신청은 201로 저장됐습니다. 기존 행사의 신청과 별도 행으로 유지됐습니다.	`tests/assignment.test.js` T05
T06 본인 대기 취소	200 · CANCELLED	본인의 PENDING 신청이 200 응답과 함께 CANCELLED로 바뀌고 CANCEL 이력이 저장됐습니다.	`tests/assignment.test.js` T06
T07 타인·운영자의 대기 취소	403 · 데이터 불변	타인 또는 운영자의 대기 취소는 403으로 거부됐고 대상 신청과 이력은 변하지 않았습니다.	`tests/assignment.test.js` T07
T08 담당자 직접 승인	200 · APPROVED	해당 행사 담당 운영자의 승인은 200으로 처리됐고 신청은 APPROVED, 승인 인원은 1명으로 확인됐습니다.	`tests/assignment.test.js` T08

거부 사례 상세

학생 Alice가 행사 1에 신청한 뒤 Charlie가 같은 신청 ID로 대기 취소를 요청하는 경우를 확인했습니다. 요청자는 신청자가 아니므로 서버는 403 `FORBIDDEN`을 반환했습니다. 요청 전후 신청은 PENDING 상태로 동일했고 CANCEL 이력도 추가되지 않았습니다. 권한 판단은 화면이 아니라 로그인 토큰에서 확인한 사용자 ID와 DB 신청자 ID를 비교해 수행합니다. 관련 테스트는 T07이며, 권한·거부 후 불변 검증은 T26에도 포함되어 있습니다.

4 검증 결과 T09부터 T16

HTTP와 code, 재조회한 상태·승인 인원·이력 건수와 내용을 기록합니다. 미실행은 그대로 표시하고 증거 파일 또는 그림 번호를 연결하세요.

ID와 상황	예상 결과	실제 결과와 데이터	증거 위치
T09 학생·비담당자의 승인	403 · 데이터 불변	학생 또는 다른 행사 담당 운영자의 승인은 403으로 거부됐고 신청 상태와 이력은 변하지 않았습니다.	`tests/assignment.test.js` T09
T10 대기 취소 후 승인	409 · CANCELLED 유지	취소된 신청을 오래된 화면에서 승인 요청해도 409 INVALID_STATE였고 CANCELLED가 유지됐습니다.	[`tests/assignment.test.js` T10 · `screenshots/U02-stale-manager-approval-rejected.png`
T11 승인 후 재승인·학생 취소	409 · APPROVED 유지	이미 승인된 신청의 재승인과 학생 취소는 409로 거부됐고 APPROVED 상태가 유지됐습니다.	`tests/assignment.test.js` T11
T12 반복 대기 취소	409 · 상태 유지	이미 취소된 신청의 반복 취소는 409 INVALID_STATE로 거부됐고 추가 이력이 없었습니다.	`tests/assignment.test.js` T12
T13 승인 정원 초과	409 · PENDING 유지	정원이 가득 찬 상태에서 다음 승인은 409 CAPACITY_FULL로 거부됐습니다. 대상 신청은 PENDING으로 남고 이력도 추가되지 않았습니다.	`tests/assignment.test.js` T13
T14 권한·대상·상태 순서	404 / 403	없는 신청은 404, 권한 없는 운영자의 행사 조회·변경 요청은 403으로 확인했습니다.	`tests/assignment.test.js` T14
T15 ID 형식·타인 상세 조회	400 / 403	잘못된 ID 형식은 400, 권한이 없는 사용자의 타인 상세 조회는 403으로 거부됐습니다.	`tests/assignment.test.js` T15
T16 DB 재연결	신청·상태 유지	DB를 닫았다 다시 연결한 후 신청 데이터와 APPROVED 상태가 유지됐습니다.	`tests/assignment.test.js` T16

정원과 최신 상태

T10에서는 운영자 화면에 남아 있는 PENDING 상태를 이용해 승인을 요청했지만 서버는 DB에서 최신 상태를 읽어 CANCELLED임을 확인하고 409로 거부했습니다. T13에서는 승인 정원 2명을 채운 뒤 추가 승인 요청을 보내 409 CAPACITY_FULL을 확인했고, 대상의 PENDING 상태와 기존 이력을 보존했습니다. 상태·권한·정원 검사는 `src/application-service.js`의 변경 트랜잭션 안에서 수행합니다.

5 검증 결과 T17부터 T24

HTTP와 code, 재조회한 상태·승인 인원·이력 건수와 내용을 기록합니다. 미실행은 그대로 표시하고 증거 파일 또는 그림 번호를 연결하세요.

ID와 상황	예상 결과	실제 결과와 데이터	증거 위치
T17 대기자 없는 승인 취소	200 · REVOKED · promoted=null	200 응답. 대상은 REVOKED, `promoted`는 `null`, 행사 승인 인원은 0명으로 확인됐습니다.	`tests/assignment.test.js` T17 · `screenshots/U05-no-waiter-empty-capacity.png`
T18 대기 등록 시각 순서	가장 이른 같은 행사 대기자 승급	같은 행사 PENDING 중 등록 시각이 가장 빠른 신청이 APPROVED로 승급됐습니다.	`tests/assignment.test.js` T18
T19 대기 등록 시각 동일	작은 ID부터 승급	등록 시각이 같은 경우 ID가 작은 신청이 먼저 선택됐습니다.	`tests/assignment.test.js` T19
T20 취소자·다른 행사 제외	해당 신청은 승급하지 않음	다른 행사 신청과 CANCELLED 신청은 후보에서 제외되고, 같은 행사의 PENDING 신청만 조회됐습니다.	`tests/assignment.test.js` T20
T21 승인 취소 권한	403 / 401 · 전체 불변	비담당 운영자 요청은 403, 인증 누락은 401로 거부됐습니다. 상태와 이력은 변하지 않았습니다.	`tests/assignment.test.js` T21
T22 반복·잘못된 상태의 승인 취소	409 · 전체 불변	APPROVED가 아닌 신청 또는 반복 승인 취소는 409로 거부됐고 상태·이력 모두 유지됐습니다.	`tests/assignment.test.js` T22
T23 승인 취소 사유 검사	400 또는 trim된 사유 저장	사유 누락·잘못된 자료형·길이 범위 초과·추가 필드는 400으로 거부됐습니다. 유효한 사유는 앞뒤 공백을 제거해 저장했습니다.	`tests/assignment.test.js` T23
T24 신청·취소·승인 이력	정확한 행위자·전후 상태·시각	APPLY·CANCEL·APPROVE 이력의 신청 ID, 행위자, 전후 상태와 생성 시각을 확인했습니다.	`tests/assignment.test.js` T24

대기 순서

`repository.js`의 `nextPending(eventId)`는 다음 SQL 정렬 기준으로 한 건만 고릅니다.

```
ORDER BY a.created_at ASC, a.id ASC
LIMIT 1
```

T18은 서로 다른 등록 시각에서 빠른 대기자가 선택되는 것을 확인했고, T19는 시각이 같을 때 ID가 작은 신청이 선택되는 것을 확인했습니다. T20은 다른 행사 및 CANCELLED 신청이 승급 후보에서 제외됨을 확인했습니다.

6 검증 결과 T25부터 T32

HTTP와 code, 재조회한 상태·승인 인원·이력 건수와 내용을 기록합니다. 미실행은 그대로 표시하고 증거 파일 또는 그림 번호를 연결하세요.

ID와 상황	예상 결과	실제 결과와 데이터	증거 위치
T25 승인 취소·승급 이력 연결	AUTO_APPROVE의 원인 ID 일치	AUTO_APPROVE의 `causeHistoryId`가 REVOKE 이력 ID와 같고, 행위자는 승인 취소를 수행한 운영자로 확인됐습니다.	`tests/assignment.test.js` T25
T26 거부 요청의 이력	추가 이력·상태 변경 없음	거부된 요청 전후 신청·이력 스냅샷이 같았습니다.	`tests/assignment.test.js` T26
T27 이력 조회 권한	본인·담당자만 200	신청자 본인과 담당 운영자는 200, 타인·비담당자는 403, 미인증 사용자는 401이었습니다.	`tests/assignment.test.js` T27
T28 신청 이력 저장 실패	500 · 신청 포함 전체 롤백	APPLY 이력 INSERT에 실패를 주입했습니다. 신청과 이력 전체가 요청 전 스냅샷과 같았고, 주입 제거 후 재시도는 201이었습니다.	`tests/assignment.test.js` T28 · `docs/evidence/rollback-verification.md`
T29 자동 승급 이력 저장 실패	500 · 두 상태·두 이력 롤백	AUTO_APPROVE 이력 INSERT에 실패를 주입했습니다. REVOKE 상태와 이력, 승급 신청 상태까지 모두 요청 전과 같았습니다. 주입 제거 후 재시도와 승급이 성공했습니다.	`tests/assignment.test.js` T29 · `docs/evidence/rollback-verification.md`
T30 나머지 이력 저장 실패	500 · 상태와 이력 롤백	APPROVE·CANCEL·REVOKE 이력 INSERT 실패를 각각 주입했고, 각 요청에서 신청 상태와 이력이 요청 전 스냅샷과 같았습니다.	`tests/assignment.test.js` T30 · `docs/evidence/rollback-verification.md`
T31 재연결 후 이력 보존	REVOKED·승급·원인 관계 유지	DB 재연결 뒤 신청·이력 데이터가 유지됐고, 승인 취소와 자동 승급의 원인 ID 연결도 보존됐습니다.	`tests/assignment.test.js` T31
T32 연속·반복 승인 취소	요청당 최대 1명 승급 · 중복 없음	연속 승인 취소마다 최대 한 명이 승급했습니다. 종료 상태에 대한 반복 요청은 거부되고 중복 승급 이력이 없었습니다.	`tests/assignment.test.js` T32

롤백 증거

T29는 `application_history`에 AUTO_APPROVE 이력을 넣는 시점에 SQLite 트리거로 실패를 발생시켰습니다. 요청 전후 스냅샷을 비교한 결과 승인 취소 대상은 기존 APPROVED 상태로, 대기자는 기존 PENDING 상태로 남았으며 REVOKE와 AUTO_APPROVE 이력도 추가되지 않았습니다. 트리거를 제거한 뒤 같은 흐름을 재시도해 정상 승인 취소와 대기자 승급이 되는 것을 확인했습니다.

T28과 T30에서도 이력 INSERT 실패를 주입했습니다. 상태 변경과 이력이 함께 롤백되는 것을 확인했으며, 테스트 출력의 `TEST_INJECTED_FAILURE`는 이 검증을 위해 의도적으로 발생시킨 오류입니다.

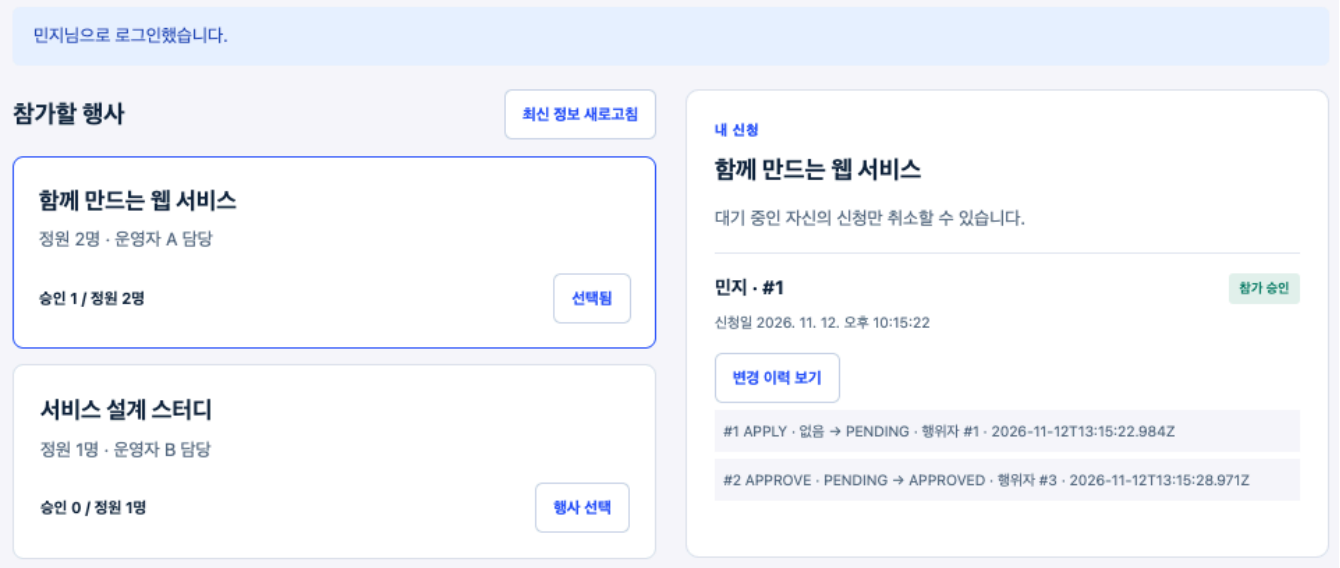
선택 심화 동시성 검증

`npm run test:advanced`는 같은 SQLite DB를 사용하는 서버 프로세스 두 개에 마지막 한 자리의 동시 승인을 요청하는 시나리오를 20회 반복했습니다. 매회 200 한 건과 409 `CAPACITY_FULL` 한 건, APPROVED 한 건과 PENDING 한 건을 확인했습니다.

추가 두 서버 검증은 동일 학생의 중복 신청 경쟁과 동일 신청의 승인 취소 경쟁을 각각 10회 실행했습니다. 중복 신청은 매회 201과 409가 한 건씩 반환됐고, 승인 취소 경쟁은 200과 409가 한 건씩 반환됐습니다. 서로 다른 학생 두 명의 동시 신청은 모두 성공했으며 SQLite `PRAGMA integrity_check` 결과는 ok였습니다. 이 결과는 해당 검사 환경과 시나리오에서의 검증이며 모든 실행 환경에서 경쟁 문제가 없음을 증명하지는 않습니다.

7 화면 검증과 실행 재현

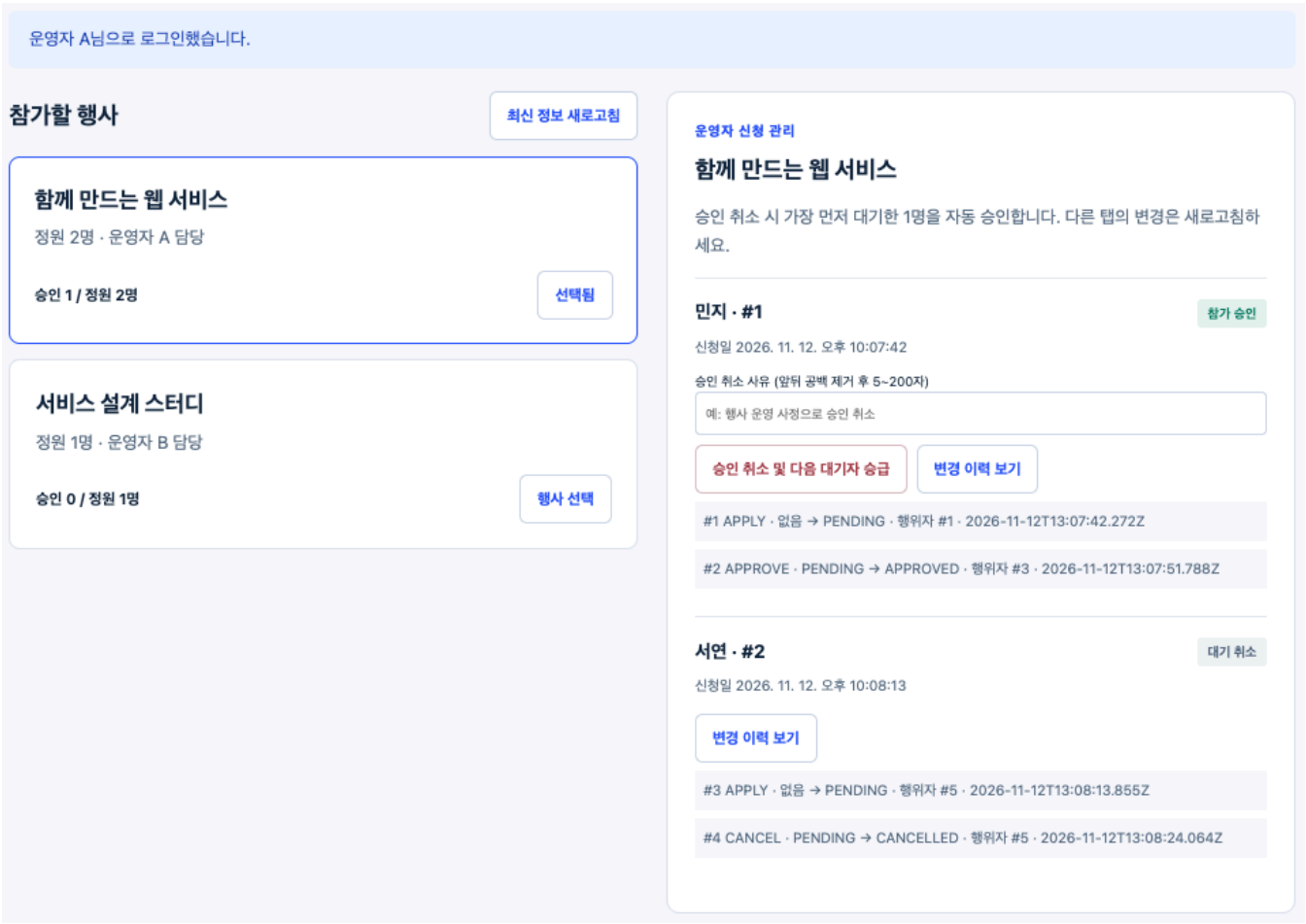
ID	흐름	결과와 증거
U01	학생 신청과 담당 운영자의 승인	Alice가 신청한 뒤 담당 운영자가 승인했습니다. 승인 상태와 행사 인원 1/2를 화면에서 확인했습니다.



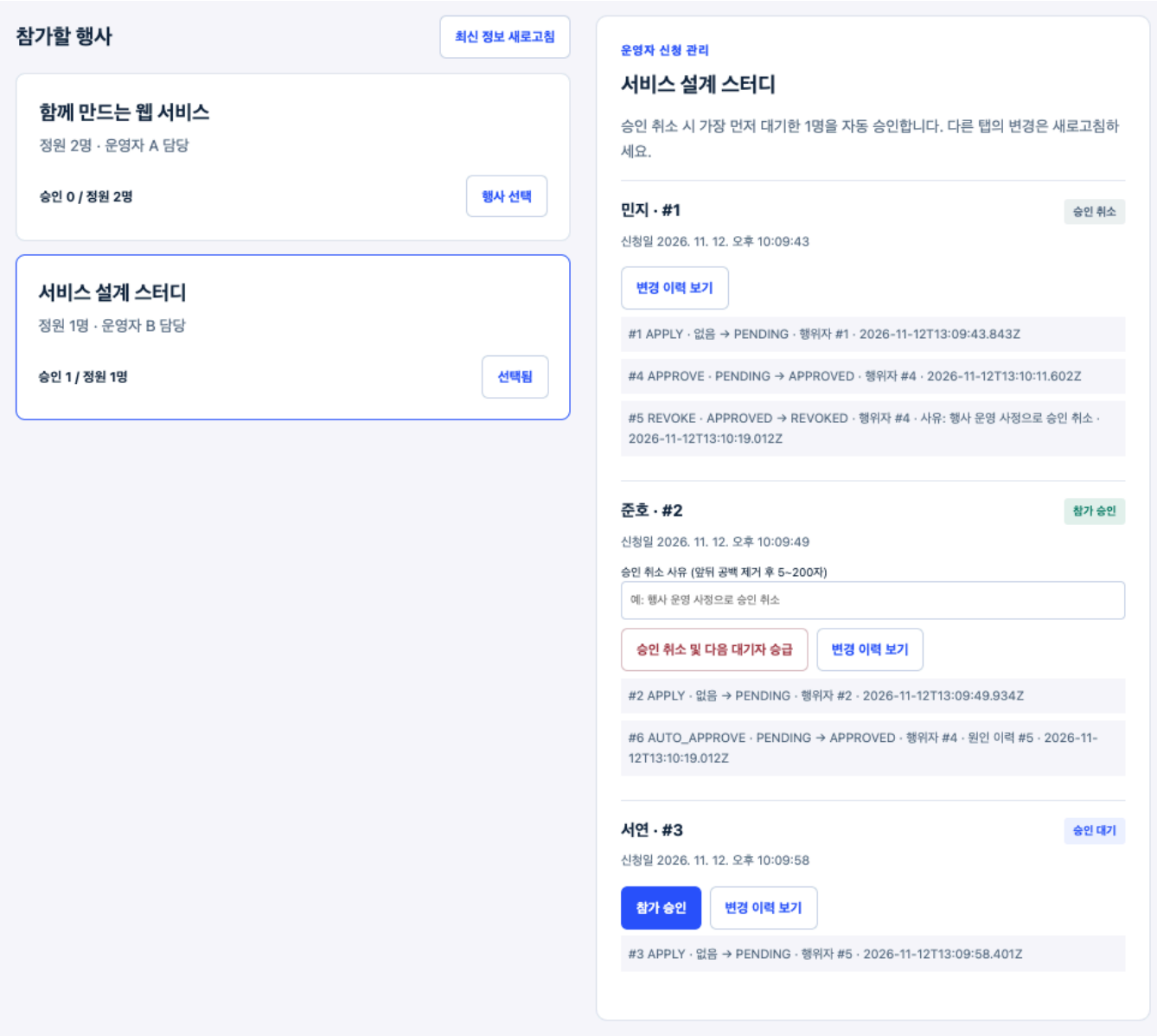
U02	학생 대기 취소 후 오래된 운영자 화면에서 승인 거부	취소된 신청을 오래된 운영자 화면에서 승인 요청했습니다. 409 INVALID_STATE가 표시되고 CANCELLED 상태가 유지됐습니다.



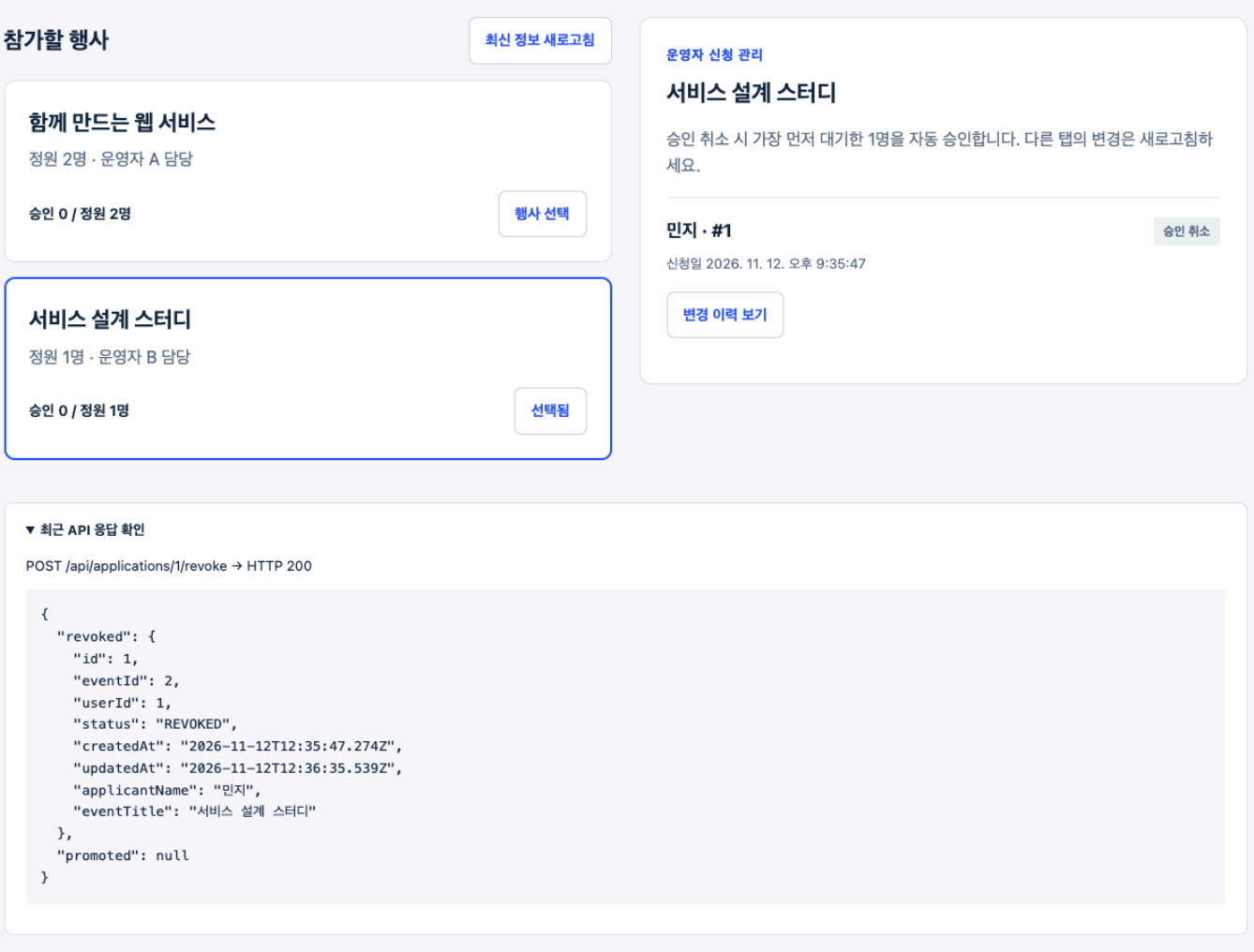
ID	흐름	결과와 증거
U03	서버 종료·재실행 후 상태와 변경 이력 유지	같은 DB를 사용해 서버를 재시작한 뒤 다시 로그인했습니다. Alice의 APPROVED, Charlie의 CANCELLED 상태와 APPLY·APPROVE 이력을 확인했습니다.



ID	흐름	결과와 증거
U04	승인 취소 → 다음 대기자 승급 → 두 신청의 이력 확인	행사 2의 승인 취소 후 Bob이 AUTO_APPROVE되고 Charlie는 PENDING으로 남았습니다. REVOKE와 AUTO_APPROVE 이력 및 원인 ID 연결을 확인했습니다.



ID	흐름	결과와 증거
U05	대기자 없는 승인 취소 → 빈자리와 REVOKED 확인	신청은 REVOKED, 승인 인원은 0/1, API 응답 `promoted`는 `null`임을 확인했습니다.



재현 절차

저장소를 복제한 뒤 저장소 루트에서 Node.js 24 환경으로 아래 명령을 실행합니다.

```
node --version
npm ci
npm test
npm run test:assignment
npm run test:advanced
npm start
```

T18은 서로 다른 등록 시각에서 빠른 대기자가 선택되는 것을 확인했고, T19는 시각이 같을 때 ID가 작은 신청이 선택되는 것을 확인했습니다. T20은 다른 행사 및 CANCELLED 신청이 승급 후보에서 제외됨을 확인했습니다.

기반 테스트는 4/4, 필수 T01~T32는 32/32, 심화 동시성 테스트는 1/1 통과했습니다. 추가 경쟁 검사는 다음 명령으로 실행했습니다.

```
node --test tests/concurrency-supplement.test.js
```

화면 접속 주소는 `http://127.0.0.1:3000`입니다. 연습 계정은 `alice`, `bob`, `charlie`, `manager1`, `manager2`이며 모두 가상 계정이고 비밀번호는 `Classroom123!`입니다. DB 초기화는 서버를 종료한 뒤 `npm run db:reset`을 실행하고 RESET을 입력합니다. 증거 문서는 `docs/evidence/`에 있으며 화면 캡처는 `docs/evidence/screenshots/`에 있습니다.

AI 활용과 참고 자료

적용 위치	직접 수정·검증한 내용	도구 또는 출처
`src/application-service.js`의 `apply`, `cancel`, `approve`, `revoke`, `transaction`	업무 규칙 검사, 상태 변경과 이력 저장 시점, 트랜잭션·롤백 구현. T01~T32와 이력 저장 실패 주입 결과를 확인했습니다.	Codex · 과제 안내문 / `docs/API.md`
`src/repository.js`의 `nextPending`, `addHistory`, `setStatus`	대기자 정렬 조회와 이력 원인 ID 저장을 구현하고 FIFO 및 원인 연결 결과를 확인했습니다.	Codex · Node.js SQLite 관련 문서
`tests/` 및 `docs/evidence/`	필수·심화 테스트를 실행하고 결과를 확인했습니다. 서버 재시작 후 데이터 유지와 U01~U05 화면 흐름을 직접 검증 및 캡처했으며, 실패 주입 후 롤백 결과를 대조했습니다.	Node.js 테스트 러너 · headless Chrome
`README.md`, `docs/evidence/*.md`	구현·실행 절차와 검증 결과를 정리했습니다.	Codex

미완성 항목과 알려진 한계

필수 과제 기능과 테스트 항목은 구현 및 검증을 완료했다. 학생 신청과 대기 신청 취소, 운영자 승인과 승인 취소, 정원 확인, 선착순 자동 승급, 변경 이력 저장 및 서버 재시작 후 데이터 유지 기능을 확인했다. 필수 테스트 T01~T32와 제공 테스트를 통과했고, 두 서버 간 동시 요청도 추가로 검증했다. 따라서 현재 남아 있는 필수 미완성 항목은 없다.

이 서비스는 과제 시연을 위한 로컬 실행형 프로토타입이다. SQLite 파일을 사용하며, 배포 환경에서 필요한 데이터베이스 백업 및 복구, 운영 모니터링, 계정 관리와 비밀번호 재설정 기능은 구현 범위에 포함하지 않았다. 로그인은 과제용 테스트 계정을 사용한다.

동시성 검증은 구현한 경쟁 요청 시나리오를 반복 실행한 결과이며, 모든 운영 환경과 부하에서의 무결성을 보장하는 것은 아니다. 실제 서비스로 확장할 경우 운영 환경에 맞는 데이터베이스 구성, 접근 통제, 백업 정책과 부하 검증이 추가로 필요하다.